[KEY] Unit 2 Coding Exercise Rubrics (2.2-2.7)

Game Programming I - Python. Internal grading reference, not for students. Each Coding Exercise is worth 10 points. Built 2026-09-28 from the actual starter files and assignment instructions in Moodle, so criteria match exactly what students are asked to do.

Submission Gate (applies to all six exercises)

Before scoring content, confirm the submission is actually viewable. This is already the dominant reason Unit 2 scores are low across the class (about 70% of all zero scores so far), so keep applying it consistently rather than case-by-case:

- CodeHS sandbox link is private / not shared "anyone with the link" → score 0, leave the standard note, allow resubmission.
- Link or file is for the wrong exercise (e.g. Booleans work submitted for the Strings assignment) → score 0, note which exercise it actually contains, allow resubmission.
- File is the untouched starter (no edits made) → score 0, note that no work was done, allow resubmission.
- Once the submission is viewable, grade it against the criteria below.

2.2 Coding Exercise: Integers (10 pts)

Starter file: GMETRIX-112-numbers.py. Student must (1) answer a fill-in-the-blank as a comment, and (2) add a print statement that reveals the data type of an existing variable.

Criteria	Points	What to check
Blank 1 answered	3	Comment answer says numbers represented as **strings/text** cannot be used in calculations. Accept close paraphrases.
Type-check statement added	4	print(type(number_of_tries)) (or equivalent using the built-in type() function) was added under step 2.
Runs and confirms the type	3	Running the file prints <class 'int'>.

2.3 Coding Exercise: Floats (10 pts)

Starter file: GMETRIX-113-numbers.py. Student adds a print statement for addition, answers a blank, changes it to division, answers another blank, then fills in a final vocabulary blank.

Criteria	Points	What to check
Addition print statement	2	Prints number_of_tries + multiplier (or equivalent) and runs without error.
Blank 3 answered	1	Answer is **float** (int + float returns float).
Changed to division	2	Same print statement now divides instead of adds, and runs without error.
Blank 5 answered	1	Answer is **float** (Python 3's / always returns a float).
Blank 6 answered	2	Answer identifies scores/counters → **integers**, money → **floats**.
Both runs verified	2	Student actually ran the file both times (not just wrote the code) - correct output present for both the addition and division version.

2.4 Coding Exercise: Strings (10 pts)

Starter file: GMETRIX-111-str.py. Student prints a full name, then answers two conceptual questions about quoting.

Criteria	Points	What to check
Full-name print statement	4	Outputs first_name and last_name together in a valid order; if last-then-first, the two are separated by a comma as instructed.
Runs without error	2	Running the file prints the full name with no errors.
Blank 4 answered	2	Answer is **curly** (or "smart") quotes - Python only accepts straight single or double quotes.
Q5 explanation	2	Identifies that the unescaped apostrophe in 'Jason's turn' closes the string early,

Criteria	Points	What to check
		breaking the rest of the line. Accept any answer that names the apostrophe as the cause.

2.5 Coding Exercise: Booleans (10 pts)

Starter file: GMETRIX-114-boolean.py. Student runs a broken file, fixes a casing bug, then answers a vocabulary blank.

Criteria	Points	What to check
Bug fixed	4	char_life = true changed to char_life = True (correct Python capitalization).
Runs correctly	3	Running the file prints "you are still in the game" with no errors.
Blank 3 answered	3	Answer is **Title** (capitalized) casing - booleans must be written True/False, not true/false.

2.6 Coding Exercise: Type Conversion (10 pts)

Starter file: GMETRIX-121-conversion.py. The bug: rating comes from input() as a string, so rating * 100 repeats the text instead of doing math, and the final print concatenates that garbled string. Fix requires converting the input and updating the print line to match.

Criteria	Points	What to check
Input converted to a real number	4	rating = int(input(...)) (or float()) so rating * 100 is real arithmetic, not string repetition.
Final print() fixed	3	Print statement updated to work with points now being a number - e.g. str(points), an f-string, or a comma-separated print. No TypeError on run.
Tested with 2+ inputs	3	Student ran the file at least twice with different valid numbers (e.g. 1 and 5) and

Criteria	Points	What to check
		confirmed real math both times, per the assignment instructions.

2.7 Coding Exercise: Reading Code with Variables (10 pts)

Starter file: score_tracker_debug.py. The bug: each round reassigns score instead of adding to it, so the "running total" never actually accumulates. This is a pure debugging task - no fill-in-the-blank questions.

Criteria	Points	What to check
Fix applied	5	Each round's score line uses accumulation (score += round_x or score = score + round_x) instead of straight reassignment.
Correct output	5	Running the file prints exactly: After round 1: 10 After round 2: 25 After round 3: 30